Peter Wolf
<Software Developer & Advisor />

React Testing mit Jest + React Testing Library

Hinweis

Bei Vite-Projekten nutzt man heute oft Vitest, aber für eine Jest-Schulung ist das Konzept identisch.

Installation

npm I @babel/preset-env @babel/preset-react jest-environment-jsdom jest @testing-library/jest-dom @testing-library/react @testing-library/user-event

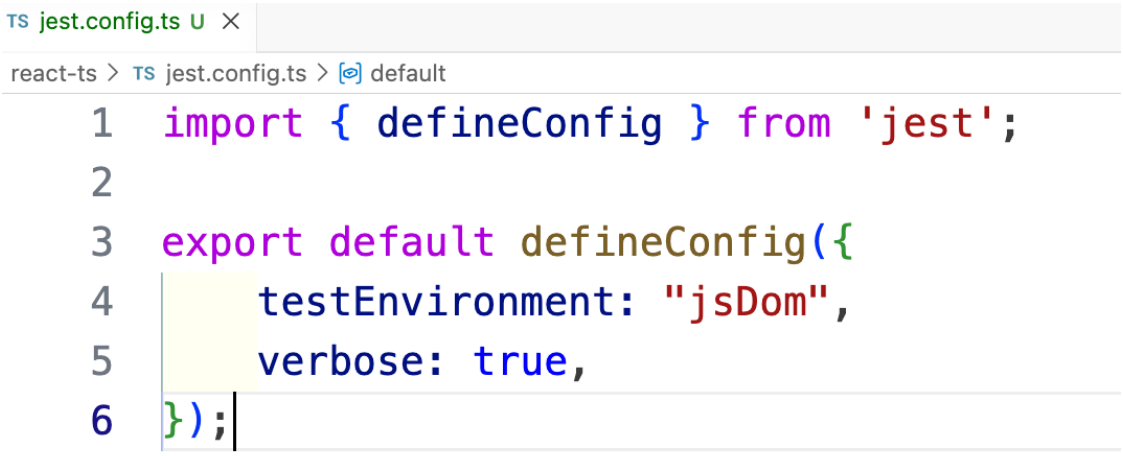
```
{
  "name": "react-testing-demo",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "vite",
    "build": "vite build",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  },
  "dependencies": {
    "react": "^19.1.0",
    "react-dom": "^19.1.0"
  },
  "devDependencies": {
    "@testing-library/jest-dom": "^6.6.3",
    "@testing-library/react": "^16.3.0",
    "@testing-library/user-event": "^14.6.1",
    "@types/jest": "^30.0.0",
```

diese infos müssen in der package.json drin stehen

```
"devDependencies": {
  "@babel/preset-env": "^7.29.7",
  "@babel/preset-react": "^7.29.7",
  "@babel/preset-typescript": "^7.29.7",
  "@eslint/js": "^10.0.1",
  "@testing-library/jest-dom": "^6.9.1",
  "@testing-library/react": "^16.3.2",
  "@testing-library/user-event": "^14.6.1",
  "@types/jest": "^30.0.0",
  "@types/node": "^24.12.3",
  "@types/react": "^19.2.14",
  "@types/react-dom": "^19.2.3",
  "@vitejs/plugin-react": "^6.0.1",
  "eslint": "^10.3.0",
  "eslint-plugin-react-hooks": "^7.1.1",
  "eslint-plugin-react-refresh": "^0.5.2",
  "globals": "^17.6.0",
  "jest": "^30.4.2",
  "jest-environment-jsdom": "^30.4.1",
  "typescript": "~6.0.2",
  "typescript-eslint": "^8.59.2",
  "vite": "^8.0.12"
}
```

Peter Wolf
<Software Developer & Advisor />

jest.config.ts



Vite.config.js

```
plugins: [react()],
esbuild: {
  include: /.*\.jsx?$/, // Treat .js and .jsx files as JSX
  exclude: [],
},
optimizeDeps: {
  esbuildOptions: {
    loader: {
      '.js': 'jsx', // Ensure dependencies with JSX in .js are handled
    },
  },
},
```

.babelrc hinzufügen in root verzeichnis

```
{
    "presets": [
        "@babel/preset-env",
        [
            "@babel/preset-react",
            {
                "runtime": "automatic"
            }
        ]
    ]
}
```

eslint.configl.js aktualisieren

```
export default defineConfig([
  globalIgnores(['dist']),

  {
    files: ['**/*.{js,jsx}'],      You, 4 days ago • firs
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },

  {
    files: ['**/*.test.{js,jsx}', '**/*.spec.{js,jsx}'],
    languageOptions: {
      globals: {
        ...globals.browser,
        ...globals.jest,
      },
    },
  },
])
```

Peter Wolf
<Software Developer & Advisor />

Einfacher Test einer Komponenten

Wir erstellen eine Komponente **Greeting.tsx**

```tsx
// Greeting.tsx
type GreetingProps = {
  name: string;
};

export default function Greeting({ name }: GreetingProps) {
  return <h1>Hallo {name}</h1>;
}
```

Einfacher Render Test **Greeting.test.tsx**

```tsx
// Greeting.test.tsx
import { render, screen } from "@testing-library/react";
import "@testing-library/jest-dom";
import Greeting from "./Greeting";

test("zeigt den Namen an", () => {
  render(<Greeting name="Peter" />);

  expect(screen.getByText("Hallo Peter")).toBeInTheDocument();
});
```

Testen einer user interaction

Erstellen einer Komponente mit einem Button

```tsx
// Counter.tsx
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>

      <button onClick={() => setCount(count + 1)}>
        Erhöhen
      </button>
    </div>
  );
}
```

Peter Wolf
<Software Developer & Advisor />

Testen des Button Clicks

```tsx
// Counter.test.tsx
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import "@testing-library/jest-dom";
import Counter from "./Counter";

test("erhöht den Counter nach Klick", async () => {
  render(<Counter />);

  expect(screen.getByText("Count: 0")).toBeInTheDocument();

  await userEvent.click(screen.getByRole("button", { name: "Erhöhen" }));

  expect(screen.getByText("Count: 1")).toBeInTheDocument();
});
```

Testen eines InputFeldes

Wir erstellen ein Inputfeld

```tsx
import { useState } from "react";

export default function LoginForm() {
  const [email, setEmail] = useState("");

  return (
    <form>
      <label htmlFor="email">E-Mail</label>

      <input
        id="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />

      <p>Eingabe: {email}</p>
    </form>
  );
}
```

Testen des Inputs

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import "@testing-library/jest-dom";
import LoginForm from "./LoginForm";

test("aktualisiert den E-Mail-Wert", async () => {
  render(<LoginForm />);

  const input = screen.getByLabelText("E-Mail");

  await userEvent.type(input, "test@example.com");

  expect(screen.getByText("Eingabe: test@example.com")).toBeInTheDocument();
});
```

Testen eines Submits

```
type SearchFormProps = {
  onSearch: (value: string) => void;
};

import { useState } from "react";

export default function SearchForm({ onSearch }: SearchFormProps) {
  const [query, setQuery] = useState("");

  function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    onSearch(query);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="query">Suche</label>
```

```
      <input
        id="query"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />

      <button type="submit">Suchen</button>
    </form>
  );
}
```

Peter Wolf
<Software Developer & Advisor />

Testing mit Mock-Funktion

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import SearchForm from "./SearchForm";

test("ruft onSearch mit dem eingegebenen Wert auf", async () => {
  const onSearchMock = jest.fn();

  render(<SearchForm onSearch={onSearchMock} />);

  await userEvent.type(screen.getByLabelText("Suche"), "React");
  await userEvent.click(screen.getByRole("button", { name: "Suchen" }));

  expect(onSearchMock).toHaveBeenCalledWith("React");
});
```